

Projekt: Flappy Bird

1 Projektbeschreibung

Im Rahmen des Moduls Grundlagen der Programmierung 2 wurde das Spiel Flappy Bird in Java mit Swing entwickelt. Der Spieler steuert einen Vogel, der durch Drücken der Leertaste nach oben beschleunigt wird. Ohne Eingabe zieht die Schwerkraft den Vogel nach unten. Das Ziel ist es, so lange wie möglich durch die Lücken der Säulen-Paare zu fliegen, ohne diese oder den Boden zu berühren. Für jedes erfolgreich passierte Säulen-Paar erhält der Spieler einen Punkt. Der bisher höchste Punktestand wird als Highscore festgehalten und nach jedem Spieldurchlauf angezeigt.

Das Spiel besteht aus drei Zuständen: dem Startbildschirm mit Vogel-Figur und START-Button, dem laufenden Spielbetrieb sowie dem Game-Over-Bildschirm. Über die Leertaste wird zwischen diesen Zuständen gewechselt.

2 Spielregeln und Funktionen

Zu Beginn erscheint der Startbildschirm. Das Spiel startet erst nach dem ersten Drücken der Leertaste. Während des Spiels gelten folgende Regeln:

- Die Leertaste versetzt den Vogel in eine Aufwärtsbewegung. Ohne Eingabe fällt er nach unten.
- Drei Säulen-Paare scrollen von rechts nach links über das Spielfeld. Die Lücke jedes Paares ist zufällig positioniert.
- Berührt der Vogel eine Säule oder den Boden, wechselt das Spiel in den Game-Over-Zustand.
- Pro passiertem Säulen-Paar erhält der Spieler einen Punkt. Der aktuelle Score wird oben im Spielfeld angezeigt.
- Im Game-Over-Bildschirm werden Score und Highscore angezeigt. Per Leertaste wird neu gestartet.

3 Architektur & Klassenstruktur

Das Projekt ist modular in vier Packages gegliedert, um Darstellung, Spiellogik, Steuerung und Speicherung klar voneinander zu trennen.

- ansicht – SpielFenster erstellt das JFrame mit fester Größe (500×700 px). SpielPanel ist das zentrale JPanel und enthält den Game Loop, die Darstellung aller Objekte sowie die Tastatureingabe.
- modell – Vogel speichert Position, Größe und vertikale Geschwindigkeit. Säule speichert Position, Breite, Lückenstart und -höhe sowie ein Flag für die Punktvergabe. SpielModell verwaltet Punktestand, Highscore und Game-Over-Status.

- logik – SpielLogik enthält die gesamte Spiellogik: Gravitation, Bewegung des Vogels, Säulen-Recycling, Punkvergabe und Kollisionserkennung.
- speicherung – HighscoreSpeicherung lädt und speichert den Highscore dauerhaft in einer Textdatei (highscore.txt). HighscoreServer öffnet Port 12345, empfängt den Punktestand des Clients, vergleicht ihn mit dem gespeicherten Highscore und schickt den höheren Wert zurück. HighscoreClient baut die TCP-Verbindung auf, sendet den Score als Zahl und empfängt den aktuellen Highscore.
- Alle Grafiken werden aus dem Ordner src/Bilder/ geladen: hintergrund.png, vogel.png, saeuleOben.png und saeuleUnten.png.

4 Spielmechanik & Kollision

Der Game Loop wird durch einen javax.swing.Timer mit einem Intervall von 16 ms (ca. 60 FPS) gesteuert. Pro Tick wird die vertikale Geschwindigkeit des Vogels um 0,5 erhöht. Ein Leertasten-Druck setzt die Geschwindigkeit sofort auf -8 px, wodurch der Vogel nach oben steigt. Erreicht der Vogel den unteren Bildschirmrand, endet das Spiel.

Die Säulen bewegen sich pro Tick um 3 px nach links. Verlässt eine Säule den Bildschirm links, wird sie rechts außerhalb des sichtbaren Bereichs neu positioniert und erhält eine neue zufällige Lückenposition. Ein Punkt wird vergeben, sobald die rechte Kante einer Säule die x-Position des Vogels passiert. Das Flag punktGegeben verhindert eine Mehrfachvergabe.

Die Kollisionserkennung erfolgt über Rectangle.intersects(). Der Hitbox des Vogels ist kleiner als das Sprite (25 px Abstand auf jeder Seite), damit das Spielgefühl fair bleibt.

5 Threads & Nebenläufigkeit

Der Game Loop läuft über einen javax.swing.Timer, der alle 16 ms ausgelöst wird. Dadurch werden Spiellogik und grafische Darstellung regelmäßig und unabhängig voneinander ausgeführt. Die Benutzeroberfläche bleibt jederzeit reaktionsfähig.

Der Highscore-Server läuft in einer dauerhaften while-Schleife und bearbeitet Client-Verbindungen sequenziell. Nach jedem Game Over sendet der HighscoreClient den Punktestand per TCP-Socket; der Server aktualisiert den Highscore bei Bedarf und schickt ihn zurück.

6 Speicherung & Netzwerkkommunikation

Der Highscore wird über HighscoreSpeicherung dauerhaft in einer Textdatei gespeichert und beim Programmstart geladen. Nach jedem Game Over sendet der HighscoreClient den aktuellen Punktestand per TCP-Socket an den HighscoreServer

(Port 12345). Das Protokoll ist textbasiert: Client schickt eine Zahl, Server antwortet mit dem höchsten bisher erreichten Wert.